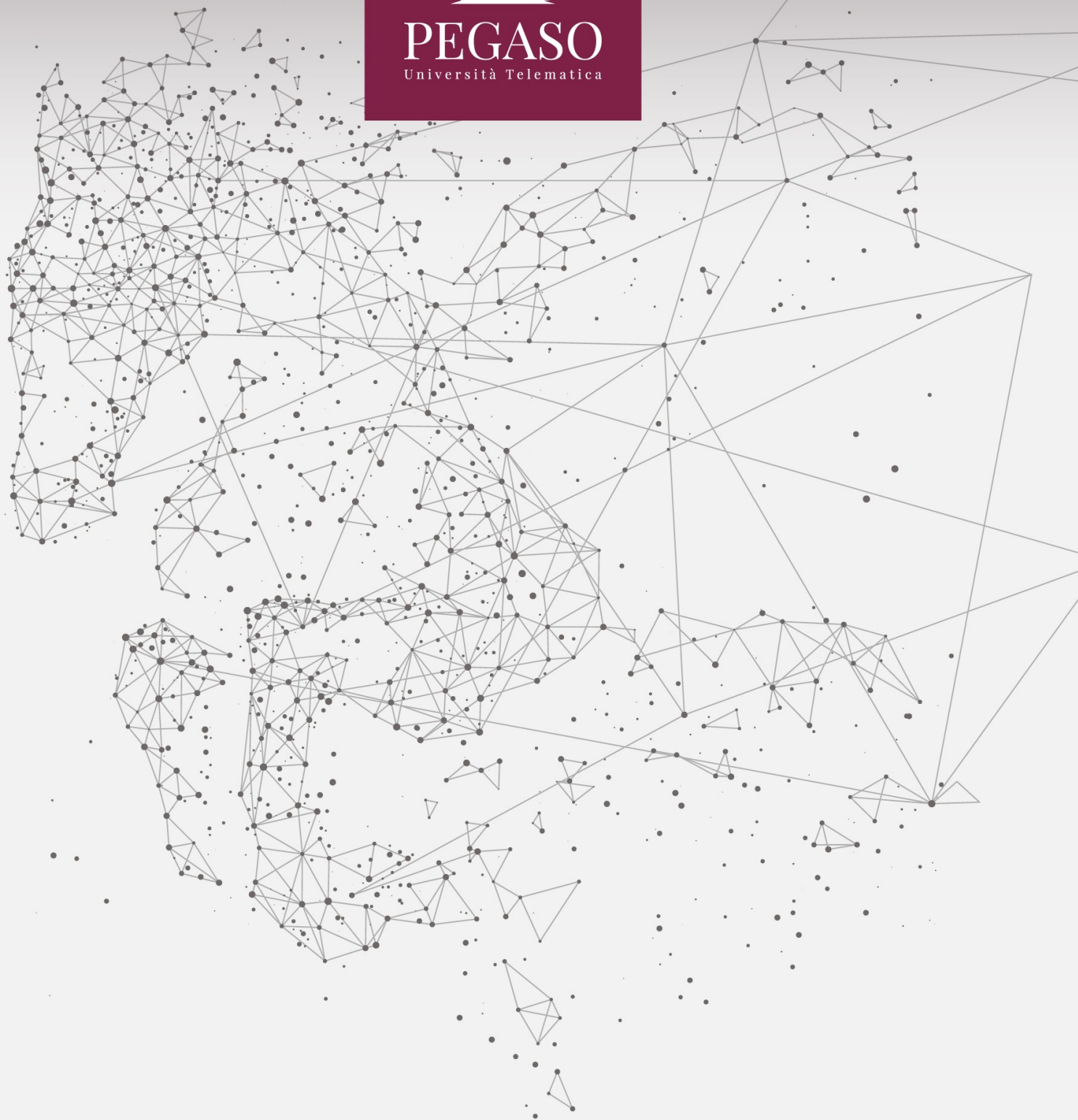




PEGASO
Università Telematica



Indice

1. IL CICLO WHILE	3
2. IL CICLO DO-WHILE (REPEAT-UNTIL).....	7
3. ESEMPI	11
BIBLIOGRAFIA	15

1. Il ciclo while

Durante lo sviluppo di un algoritmo, è comune imbattersi in situazioni in cui alcune istruzioni devono essere ripetute più volte. I blocchi di iterazione nei diagrammi di flusso sono progettati proprio per questo scopo, permettendo di definire strutture iterative. I blocchi di iterazione consentono di iterare, ovvero ripetere più volte, un insieme di istruzioni specifiche, garantendo un'esecuzione efficiente e ordinata delle operazioni necessarie all'interno dell'algoritmo.

Il ciclo while è un blocco di iterazione in cui l'esecuzione di una serie di istruzioni continua finché una condizione specificata rimane vera. Il ciclo while è caratterizzato da una condizione di ingresso o permanenza, che viene valutata prima di ogni iterazione. All'interno del ciclo, il corpo del ciclo contiene una o più azioni da eseguire o ripetere. Inoltre, può essere spesso necessaria un'inizializzazione preliminare delle variabili utilizzate nella condizione del ciclo, per garantire che la condizione possa essere correttamente valutata. Questo meccanismo permette di eseguire un insieme di istruzioni in modo ripetitivo, fintanto che la condizione specificata resta soddisfatta.

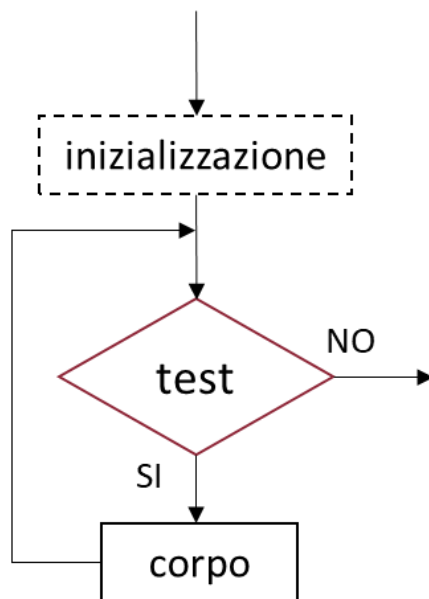


Figura 1 – Diagramma di flusso di un ciclo while generico

Come riportato in Fig. 1, in un diagramma di flusso, un ciclo while può essere realizzato utilizzando un rombo, posizionato a monte del corpo del ciclo, per rappresentare la condizione del ciclo, da cui si diramano due percorsi del flusso: i) la freccia di connessione SI per entrare nel ciclo e identificare le azioni che dovranno essere eseguite ed eventualmente ripetute (il corpo del ciclo) prima di tornare al rombo e verificare nuovamente la condizione del ciclo; ii) la freccia di connessione NO per non entrare (o per uscire) nel (dal) ciclo e proseguire il flusso di esecuzione oltre il blocco del ciclo while.

Si noti che la freccia di connessione SI, dopo aver attraversato le azioni che compongono il corpo del ciclo, ritorna al rombo che rappresenta la condizione del ciclo, realizzando graficamente un ciclo chiuso che caratterizza il blocco di iterazione.

La condizione, o test, di ingresso/permanenza in un ciclo while è una domanda o un'espressione booleana che viene verificata per determinare se entrare o rimanere nel ciclo. Si continua a iterare nel ciclo finché questa condizione risulta vera.

Affinché la condizione di un ciclo while abbia sempre un valore sensato e interpretabile, può essere necessario includere delle istruzioni di inizializzazione prima dell'inizio effettivo del ciclo, cioè prima che la condizione del ciclo venga valutata. Infatti, le istruzioni di inizializzazione permettono di garantire che le variabili coinvolte nella condizione del ciclo while siano correttamente impostate, assicurando così il corretto funzionamento del ciclo stesso.

Il corpo di un ciclo while è costituito da uno o più blocchi di istruzioni che vengono eventualmente eseguiti a ogni iterazione del ciclo. Si noti che il corpo di un ciclo while potrebbe non essere mai eseguito se la condizione del ciclo non dovesse mai risultare verificata.

Nel caso in cui la condizione del ciclo sia verificata almeno una volta, sarà invece possibile entrare nel ciclo almeno un volte ed eseguire le istruzioni del corpo del ciclo. In tal caso, è fondamentale che all'interno del corpo del ciclo sia inserita almeno un'istruzione in grado di modificare l'esito della condizione del ciclo, consentendo così che prima o poi sia possibile uscire dal ciclo. In caso contrario, il ciclo potrebbe procedere all'infinito, eseguendo le istruzioni ripetutamente senza mai riuscire a terminare.

A titolo di esempio, proviamo a definire il diagramma di flusso di un algoritmo che stampi a video i numeri da 1 a 10 utilizzando un ciclo while che ripeta 10 volte l'istruzione di stampa applicata ad una variabile il cui valore venga fatto cambiare da 1 a 10. Per impostare questo ciclo while è sufficiente utilizzare una variabile, da incrementare ad ogni esecuzione del corpo del ciclo, con cui contare le volte che si esegue il ciclo, cioè un contatore che conti da 1 a 10. A tale scopo, consideriamo le seguenti istruzioni:

1. assegna il valore 10 alla variabile n;
2. assegna il valore 1 alla variabile i;
3. **finché ($i \leq n$) fai (while $i \leq n$ do)**
4. stampa il valore della variabile i;
5. incrementa di 1 il valore di i.

Il diagramma di flusso di questo algoritmo è riportato in Fig. 2, da cui si può notare che il corpo del ciclo è costituito dalle azioni (4) e (5).

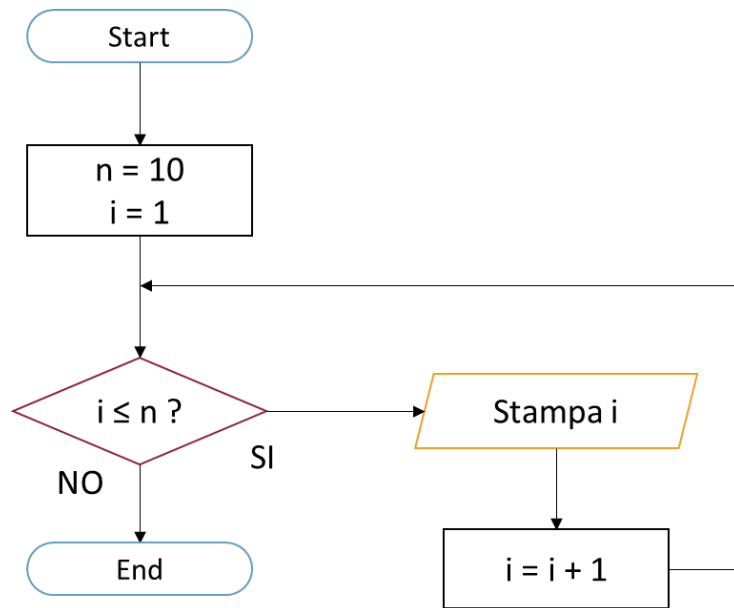


Figura 2 – Diagramma di flusso di un algoritmo per la stampa dei primi 10 numeri tramite ciclo while

Come ulteriore esempio, proviamo a definire il diagramma di flusso di un algoritmo che legga 10 numeri in ingresso, e stampi a video il valore massimo del loro valore assoluto. A tale scopo, l'algoritmo potrebbe leggere il primo numero e impostare il suo valore assoluto come valore massimo iniziale. Successivamente, l'algoritmo potrebbe utilizzare un ciclo while che ripeta 9 volte la lettura di un nuovo numero in input, il confronto tra il massimo attuale e il valore assoluto dell'ultimo numero letto in input, e ovviamente l'incremento della variabile contatore che permette di tenere traccia di quante volte il corpo del ciclo è stato eseguito.

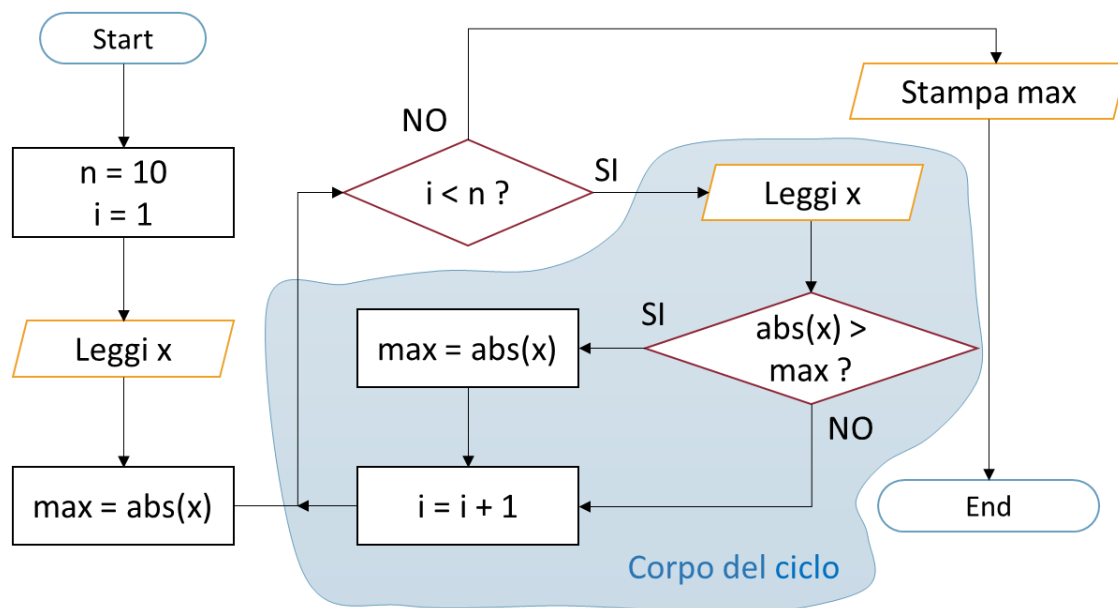


Figura 3 – Diagramma di flusso di un algoritmo per la stampa del massimo tra 10 numeri

Consideriamo quindi le seguenti istruzioni:

1. assegna il valore 10 alla variabile n , e il valore 1 alla variabile i ;
2. leggi in input un numero, ed identificalo con x ;
3. assegna alla variabile max il valore assoluto di x ;
4. **while** ($i < n$) **do**
5. leggi in input un numero, ed identificalo con x ;
6. **se** $abs(x)$ è maggiore di max **allora**
7. assegna alla variabile max il valore $abs(x)$;
8. incrementa di 1 il valore di i ;
9. stampa il valore della variabile max .

Il diagramma di flusso di questo algoritmo è riportato in Fig. 3, da cui si può notare che il corpo del ciclo è costituito dalle azioni (5), (6), (7) e (8).

2. Il ciclo do-while (repeat-until)

Il ciclo do-while, anche detto ciclo repeat-until, è un blocco di iterazione in cui l'esecuzione di una serie di istruzioni continua finché una condizione specificata rimane vera. Il ciclo do-while è caratterizzato dal fatto che le istruzioni del corpo del ciclo vengono sempre eseguite almeno una volta prima di verificare la condizione del ciclo e determinare se sia il caso o meno di ripetere le azioni del corpo del ciclo.

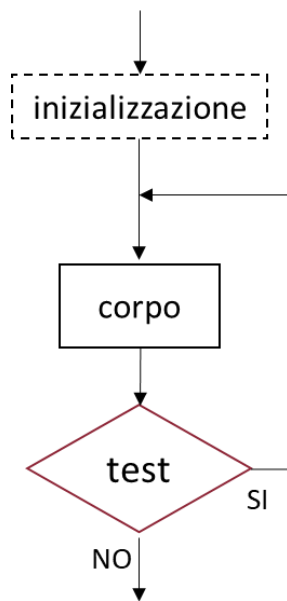


Figura 4 – Diagramma di flusso di un ciclo do-while generico

Come riportato in Fig. 4, in un diagramma di flusso, un ciclo do-while può essere realizzato utilizzando un rombo, posizionato a valle del corpo del ciclo, per rappresentare la condizione del ciclo, da cui si diramano due percorsi del flusso: i) la freccia di connessione SI per entrare nel ciclo e identificare le azioni che dovranno essere ripetute (il corpo del ciclo) tornando indietro in un punto precedente del flusso di esecuzione; ii) la freccia di connessione NO per non entrare (o per uscire) nel (dal) ciclo e proseguire il flusso di esecuzione oltre il blocco del ciclo do-while.

Si noti che la freccia di connessione SI, tornando indietro in un punto precedente del flusso di esecuzione, realizza graficamente un ciclo chiuso che caratterizza il blocco di iterazione.

La condizione, o test, di ingresso/permanenza in un ciclo do-while è una domanda o un'espressione booleana che viene verificata per determinare se entrare o rimanere nel ciclo. Si continua a iterare nel ciclo finché questa condizione risulta vera.

Affinché la condizione di un ciclo do while abbia sempre un valore sensato e interpretabile, può essere necessario includere delle istruzioni di inizializzazione prima dell'inizio effettivo del ciclo o nel corpo

del ciclo stesso, cioè prima che la condizione del ciclo venga valutata. Tali istruzioni devono garantire che le variabili coinvolte nella condizione del ciclo `do-while` siano correttamente impostate, assicurando così il corretto funzionamento del ciclo stesso.

Il corpo di un ciclo `do-while` è costituito da uno o più blocchi di istruzioni che vengono sempre eseguiti almeno una volta, ed eventualmente ripetuti a ogni iterazione del ciclo. In quest'ultimo caso, è fondamentale che all'interno del corpo del ciclo sia inserita almeno un'istruzione in grado di modificare l'esito della condizione del ciclo, consentendo così che prima o poi sia possibile uscire dal ciclo. In caso contrario, il ciclo potrebbe procedere all'infinito, eseguendo le istruzioni ripetutamente senza mai riuscire a terminare.

A titolo di esempio, proviamo a definire il diagramma di flusso di un algoritmo che stampi a video i numeri da 1 a 10 utilizzando un ciclo `do-while` che ripeta 10 volte l'istruzione di stampa applicata ad una variabile il cui valore venga fatto cambiare da 1 a 10. Per impostare questo ciclo `do-while` è sufficiente utilizzare una variabile, da incrementare ad ogni esecuzione del corpo del ciclo, con cui contare le volte che si esegue il ciclo, cioè un contatore che conti da 1 a 10. A tale scopo, consideriamo le seguenti istruzioni:

1. assegna il valore 10 alla variabile `n`;
2. assegna il valore 1 alla variabile `i`;
3. **do**
4. stampa il valore della variabile `i`;
5. incrementa di 1 il valore di `i`;
6. **while** (`i ≤ n`).

Il diagramma di flusso di questo algoritmo è riportato in Fig. 5, da cui si può notare che il corpo del ciclo è costituito dalle azioni (4) e (5).

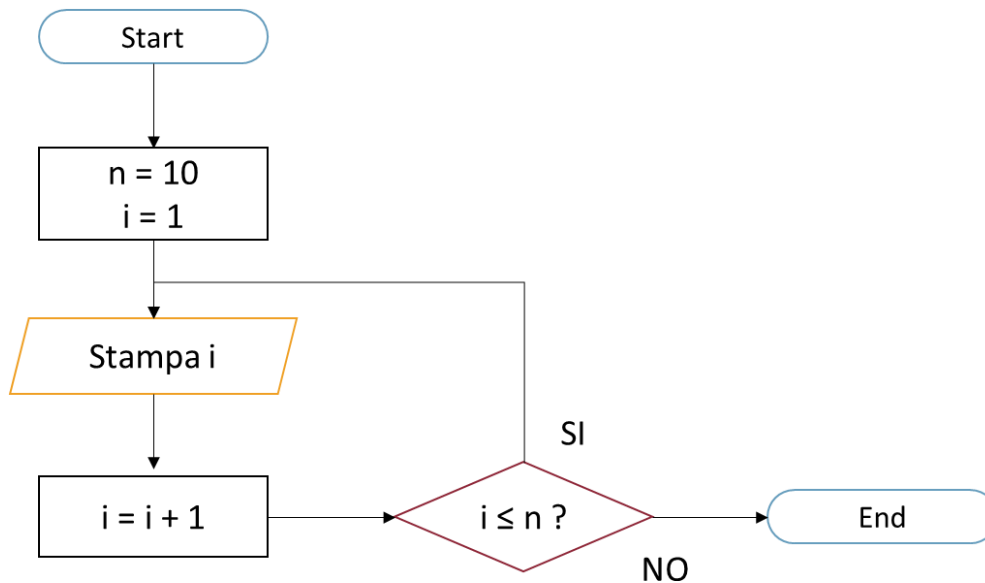


Figura 5 – Diagramma di flusso di un algoritmo per la stampa dei primi 10 numeri tramite ciclo do-while

Come ulteriore esempio, proviamo a definire il diagramma di flusso di un algoritmo che legga 10 numeri in ingresso, e stampi a video il valore massimo del loro valore assoluto.

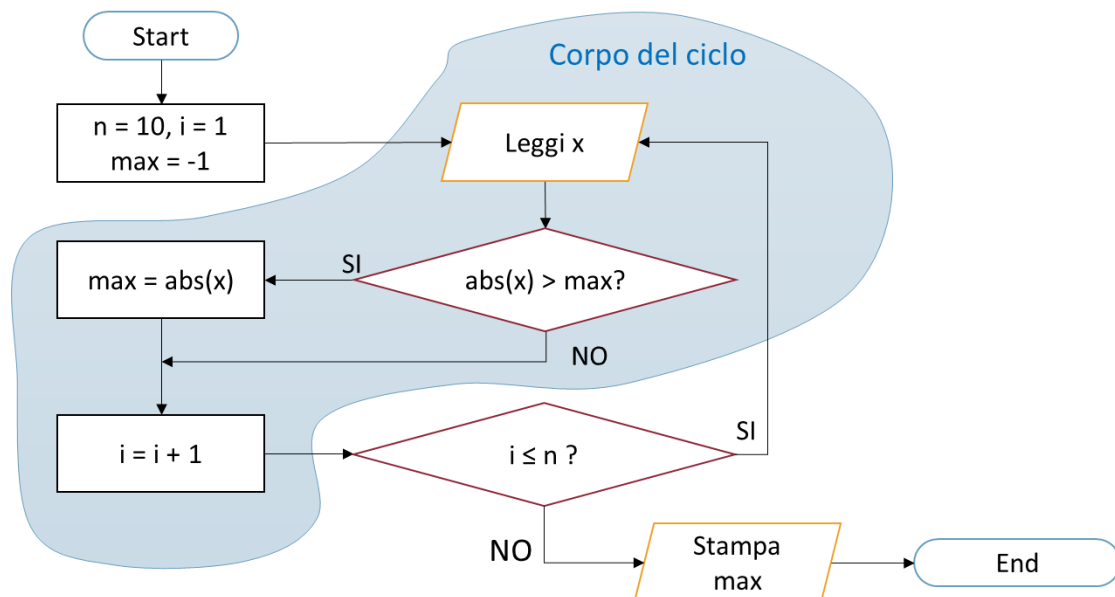


Figura 6 – Diagramma di flusso per la stampa del massimo tra 10 numeri tramite ciclo do-while

A tale scopo, l'algoritmo potrebbe impostare il valore massimo iniziale ad un valore tale da risultare sicuramente inferiore al valore assoluto del primo numero che si andrà a leggere, ad esempio pari al valore -1. Successivamente, l'algoritmo potrebbe utilizzare un ciclo do-while che ripeta 10 volte la lettura di un numero in input, il confronto tra il massimo attuale e il valore assoluto dell'ultimo numero letto in input, e ovviamente l'incremento della variabile contatore che permette di tenere traccia di quante volte il corpo del ciclo è stato eseguito.

Consideriamo quindi le seguenti istruzioni:

1. assegna il valore 10 alla variabile n, e il valore 1 alla variabile i;
2. assegna il valore -1 alla variabile max;
3. **do**
4. leggi in input un numero, ed identificalo con x;
5. **se** abs(x) è maggiore di max **allora**
6. assegna alla variabile max il valore abs(x);
7. incrementa di 1 il valore di i;
8. **while** (i≤n);
9. stampa il valore della variabile max.

Il diagramma di flusso di questo algoritmo è riportato in Fig. 6, da cui si può notare che il corpo del ciclo è costituito dalle azioni (4), (5), (6) e (7).

3. Esempi

Come primo esempio, proviamo a definire il diagramma di flusso di un algoritmo che legga un numero in input e:

- se il numero è pari, stamparlo a video;
- se il numero è dispari tornare al punto 1;
- se il numero è uguale a “-1”, terminare l’algoritmo.

Una soluzione potrebbe essere realizzata utilizzando un ciclo do-while che legga un numero in input finché il numero inserito non risulta avere valore -1. Il corpo di tale ciclo potrebbe poi occuparsi di stampare o meno il valore inserito a seconda che esso sia pari o meno. Potremmo quindi considerare le seguenti istruzioni:

1. **do**
2. leggi in input un numero, ed identificalo con N;
3. **se** N è pari **allora**
4. stampa il valore della variabile N;
5. **while** (i≠-1).

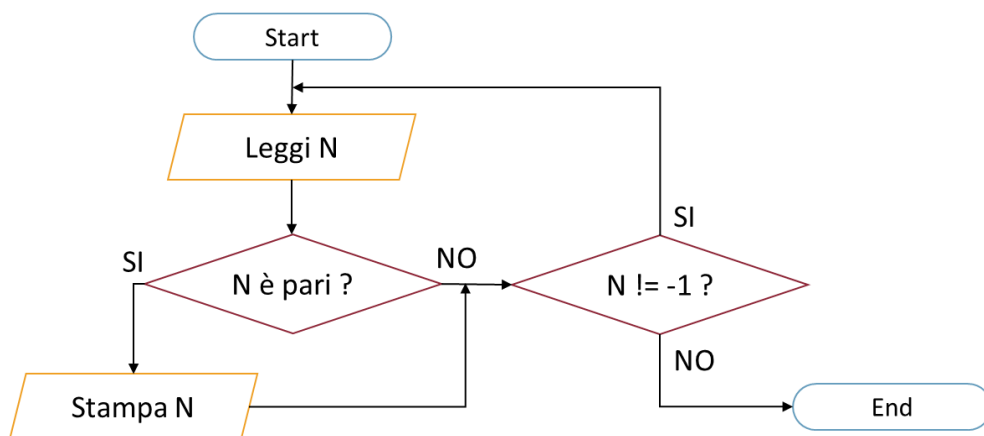


Figura 7 – Diagramma di flusso per la lettura di numeri in input finché non viene inserito il valore -1

Il diagramma di flusso di questo algoritmo è riportato in Fig. 7, da cui si può notare che il corpo del ciclo è costituito dalle istruzioni (3), (4), e (5).

Consideriamo un altro esempio, proviamo a definire il diagramma di flusso di un algoritmo che:

- legga in ingresso 5 prezzi di 5 prodotti diversi;
- stampi il prezzo finale scontato del 15% se il costo totale è minore di 500 €;
- altrimenti stampi il prezzo finale scontato del 20%.

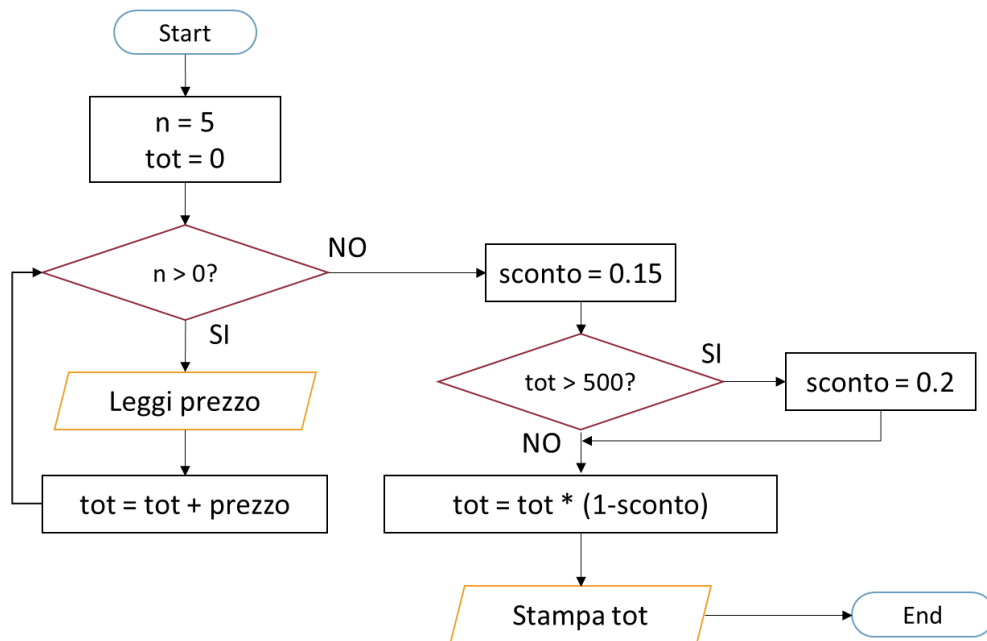


Figura 8 – Diagramma di flusso per il calcolo della somma scontata di alcuni prezzi di prodotti

Una soluzione potrebbe essere realizzata utilizzando un ciclo while che legga 5 volte in input 5 il prezzo di un prodotto, tramite ad esempio un contatore che varia da 5 a 1. Il corpo di tale ciclo potrebbe poi occuparsi di calcolare la somma dei 5 prezzi inseriti. A valle del ciclo, l'algoritmo dovrebbe utilizzare un rombo per decidere quale sconto applicare e poi stampare il totale calcolato.

Potremmo quindi considerare le seguenti istruzioni:

1. assegna il valore 5 alla variabile n, e il valore 0 alla variabile tot;
2. **while** (n>0) **do**
3. leggi in input un numero, ed identificalo con prezzo;
4. aggiungi il valore di prezzo al valore attuale della variabile tot;
5. assegna alla variabile sconto il valore 0.15;
6. **se** tot è maggiore di 500 **allora**

7. assegna alla variabile sconto il valore 0.2;
8. aggiorna il valore della variabile tot applicando lo sconto selezionato;
9. stampa il valore della variabile tot.

Il diagramma di flusso di questo algoritmo è riportato in Fig. 8, da cui si può notare che il corpo del ciclo è costituito dalle istruzioni (3) e (4).

Come ulteriore esempio, proviamo a definire il diagramma di flusso di un algoritmo che effettui la lettura in input di una serie di valori numerici non negativi:

- il programma termina quando il valore immesso è pari a zero;
- calcolare e stampare la media dei valori inseriti.

Una possibile soluzione è data dal diagramma di flusso riportato in Fig. 9.

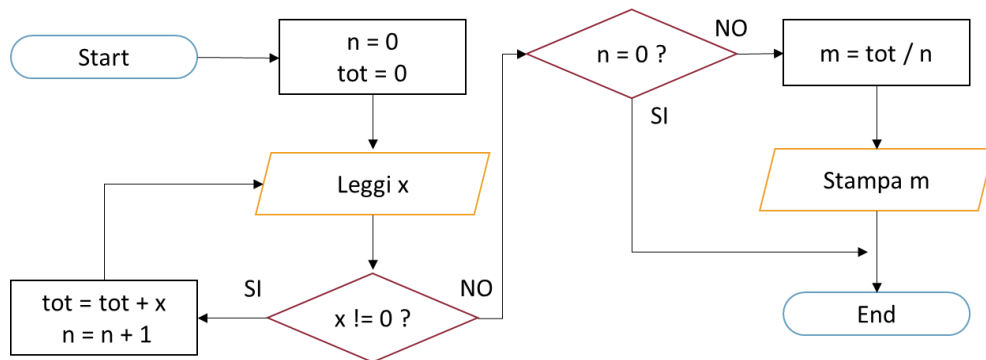


Figura 9 – Diagramma di flusso per il calcolo della media di una serie di valori numerici non negativi

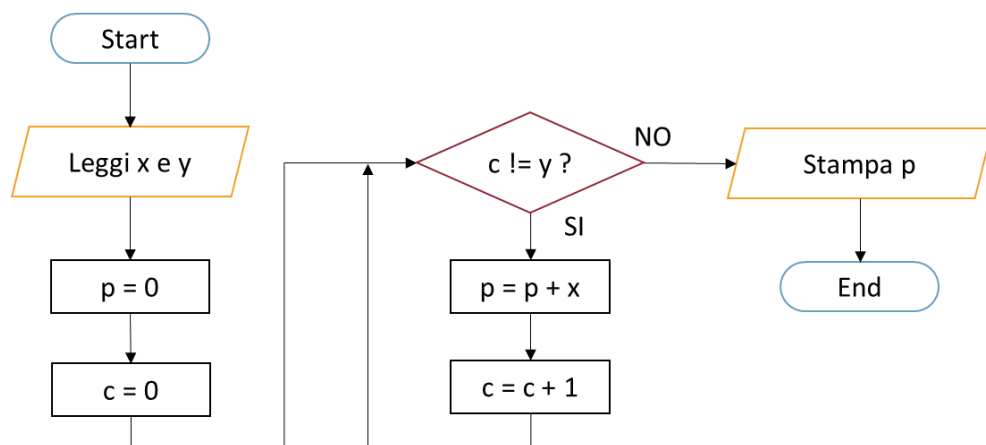


Figura 10 – Diagramma di flusso per il calcolo del prodotto tra $x > 0$ e $y > 0$, come sequenza di somme

Come ultimo esempio, proviamo a definire il diagramma di flusso di un algoritmo che calcoli e stampi il prodotto tra due numeri x e y , come sequenza di somme.

Una possibile soluzione può essere definita tramite il diagramma di flusso riportato in Fig. 10. Si noti che tale algoritmo suppone che $x > 0$ e $y > 0$.

Se invece si vuole supportare il caso in cui $x \geq 0$ e $y \geq 0$, una possibile soluzione può essere definita tramite il diagramma di flusso riportato in Fig. 11.

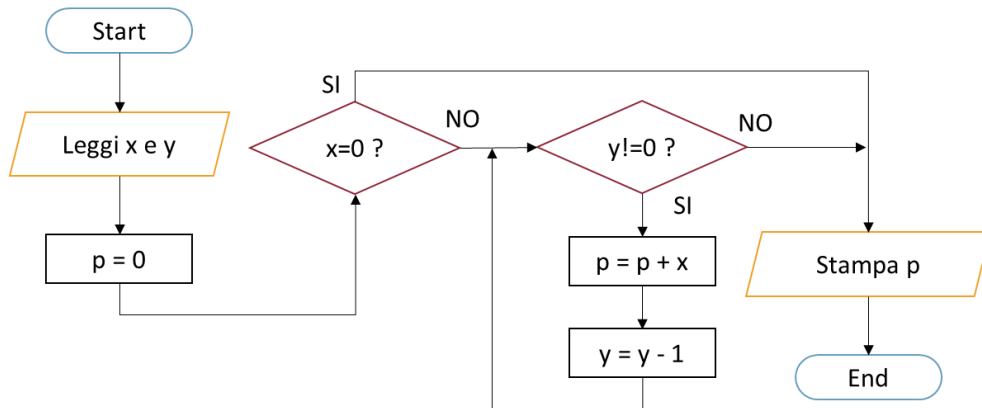


Figura 11 – Diagramma di flusso per il calcolo del prodotto tra $x \geq 0$ e $y \geq 0$, come sequenza di somme

Bibliografia

- Le radici dell'informatica. Dal bit alla programmazione strutturata. Angelo Chianese, Vincenzo Moscato, Antonio Picariello, Carlo Sansone. Maggioli Editore.